

Data Structures and Algorithms

Assignment 2

Binary Search Trees and Red-Black Trees

Prepared by:

Youssef Ali Rashedy Ali

ID: 23010999

Computer and Systems Engineering
Faculty of Engineering, Alexandria University



April 2026

Contents

1	Introduction	3
2	Implementation	3
2.1	Abstract Tree Design	3
2.2	Specialized Node Classes	4
2.3	Binary Search Tree (BST)	4
2.4	Red-Black Binary Search Tree (RBBST)	5
2.5	Validation of Red-Black Properties	6
2.6	Testing of BST and Red-Black BST	6
2.6.1	General Invariant Checks	6
2.6.2	Insertion Behavior	7
2.6.3	Deletion Behavior	7
2.6.4	Height Behavior	7
2.6.5	Clear Behavior	7
2.6.6	In-order Traversal Behavior	8
2.6.7	Regression Tests	8
2.7	Logging of Red-Black Tree Operations	8
3	Benchmarking Red-Black Tree, BST, and QuickSort	9
3.1	Input Distributions	9
3.2	Operations Benchmarked	9
3.3	Execution Protocol	10
3.4	Data Aggregation and Statistics	10
3.5	Additional Benchmark: Sorting Comparison	10
3.6	Methodology Highlights	11
4	Results and Analysis	11
4.1	Main Performance Comparison	11
4.2	Operation-Level Aggregation	12
4.3	The Impact of Input Distribution (Speedup Matrix)	12
5	Visual Analysis	13
5.1	Runtime Distribution and Stability Analysis	13
5.2	Tree Height and Structural Integrity	14
5.3	Speedup Analysis and Efficiency Thresholds	15
5.4	Operation-Specific Performance Scaling	16
5.5	Correlation Between Tree Height and Latency	17

5.6	Performance Trends Across Distributions	18
6	Comparative Sorting Analysis	19
6.1	Performance Benchmarks	20
6.2	Algorithmic Performance Analysis	20
6.2.1	The Impact of Memory Locality and Boxing	20
6.2.2	Tree-Based Sorting Trade-offs	21
6.3	Key Observations	21
6.4	Visualizing the Performance Hierarchy	22
6.4.1	Overall Ranking	24
7	Technical Implementation Considerations	24
7.1	Memory Layout and Cache Locality	24
7.2	Garbage Collection and Benchmark Integrity	24
7.3	Efficiency of Bulk Reclamation vs. Manual Deletion	25
8	TreeShell Module	25
8.1	Design Overview	26
8.2	Supported Command Style	26
8.3	State Model	26
8.4	Implementation Considerations	27
8.5	Current Limitations	27
8.6	Role in the Project	27
9	Conclusion	27

1 Introduction

This assignment focuses on the implementation and analysis of two tree-based data structures: the Binary Search Tree (BST) and the Red-Black Tree (RBT). Both structures support fundamental operations such as insertion, deletion, and search, but differ in how they maintain balance and performance.

The Binary Search Tree is simple and efficient under ideal conditions. However, its performance depends heavily on the order of input data, and it may degrade to linear time when the tree becomes unbalanced. In contrast, the Red-Black Tree introduces additional constraints using node coloring and rotations to maintain a balanced structure, ensuring logarithmic performance for all major operations.

To evaluate these differences, both data structures were implemented in Java using a unified interface. A benchmarking framework was developed to measure the performance of key operations under different input distributions, including random and nearly sorted data.

The results of this assignment provide a practical comparison between the two structures, highlighting the importance of self-balancing mechanisms in maintaining consistent performance.

2 Implementation

2.1 Abstract Tree Design

At the top of the hierarchy, an abstract class `AbstractBST` is defined to provide a unified interface and shared functionality for different tree implementations.

The class is parameterized using three generic types:

- **K**: the key type, constrained to implement `Comparable` to allow ordering.
- **V**: the value associated with each key.
- **N**: the node type, which is constrained to extend the internal `Node` class.

The use of a recursive generic definition:

$$N \text{ extends } \text{Node}<K, V, N>$$

ensures type safety by enforcing that all node instances used within the tree are consistent with the specific implementation. At the same time, it allows flexibility for extending the

node structure in specialized trees such as Red-Black Trees, where additional properties (e.g., color) are required.

This design enables code reuse across multiple tree variants while preserving strict type constraints, avoiding the need for casting or duplicated logic.

2.2 Specialized Node Classes

The generic `Node` class defined in `AbstractBST` serves as the foundation for all tree nodes. This base node is extended by two specialized node classes to support different types of trees:

- **BSTNode** — a straightforward extension used for standard binary search trees:
 - Generic type parameters: `K` for key and `V` for value.
 - Its purpose is primarily to serve as a concrete type for the generic node parameter `N`, ensuring type safety without adding extra fields.
- **RBNode** — an extension used for Red-Black trees, adding color semantics:
 - Includes an additional field `Color color` to store node color (RED or BLACK).
 - Constructors initialize new nodes to RED by default, while NIL sentinel nodes are BLACK.

The NIL node is shared across both `BSTNode` and `RBNode` classes. This consistent sentinel simplifies tree operations, avoids null checks, and minimizes the need for method overriding. By maintaining a single NIL representation, the tree logic can operate uniformly across different node types.

2.3 Binary Search Tree (BST)

The `BST` class extends `AbstractBST` and provides a concrete implementation for standard binary search trees. It does not introduce new functionality beyond what is already defined in the abstract class; instead, it focuses on **node creation**.

- **Factory Method Implementation** — `BST` implements the abstract methods:
 - `createNode(K key, V value)`: constructs a new `BSTNode` instance.
 - `createNIL()`: returns a sentinel `BSTNode` representing the NIL node.
- **Facade for Node Management** — By providing concrete nodes, the `BST` class:
 - Fixes the node type for the generic abstract tree.

- Simplifies tree operations by avoiding repeated type checks.
- Delegates all tree logic (insertion, deletion, traversal, height calculation, printing) to the abstract class.

This approach follows the **factory method pattern**, where node creation is delegated to the concrete subclass. The abstract class acts as a template and enforces the interface, while the concrete BST class provides the specific type of nodes needed.

2.4 Red-Black Binary Search Tree (RBBST)

The RBBST class extends `AbstractBST` and implements a self-balancing binary search tree using the Red-Black Tree properties. Its main responsibilities are:

- **Node Management via Factory Methods:** Implements `createNode(K key, V value)` and `createNIL()` to produce `RBNode` instances. This enforces type safety while reusing the BST logic from `AbstractBST`.
- **Red-Black Properties Enforcement:** Each `RBNode` has a `color` field (`RED` or `BLACK`) used to maintain balancing. Sentinel `NIL` nodes are always black for consistency.
- **Insertion with Fixup:** The `insert(K key, V val)` method uses `insertRaw` from the superclass and calls `fixupRB` to enforce red-black properties:
 1. Case 1: Recolor parent and uncle if both are red.
 2. Cases 2 and 3: Rotate and recolor depending on the position of the new node.

The root is always recolored black after insertion.

- **Deletion with Fixup:** The `delete(K key)` method performs BST deletion and calls `fixupDelete` to maintain red-black properties, handling double-black situations, rotations, and recoloring of sibling nodes.
- **Rotation Operations:** `leftRotate(RBNode x)` and `rightRotate(RBNode x)` are implemented to support balancing during insertions and deletions.

Summary: The RBBST class acts as both a concrete node factory and a controller for red-black balancing. While it leverages `AbstractBST` for basic tree operations, it introduces color semantics, fixup logic, rotations, and validation, making it a complete self-balancing BST implementation.

2.5 Validation of Red-Black Properties

To ensure correctness of the `RBBST` implementation, a static inner class `Validator` is used. The validator enforces the essential Red-Black Tree invariants:

- **Root is Black:** The root node must always be black. This is checked explicitly in the validator.
- **Black-Height Consistency:** For every path from the root to a leaf, the number of black nodes must be the same. The validator recursively computes the black height along each path and ensures consistency.
- **No Consecutive Red Nodes:** Red nodes cannot have red children. The validator recursively checks all nodes to ensure that no red node has a red child.

Unnecessary Checks: Some invariants do not require validation due to the design of the implementation:

- **Node Colors:** All nodes are either red or black by construction, thanks to the `Color` enum.
- **Black Leaves:** All leaves are represented by the sentinel `NIL` node, which is always black. Since the `NIL` nodes are created at the beginning and never change color, this property is inherently maintained.

Summary: By combining recursive checks for black-height and red-red violations with the root color check, the `Validator` ensures that the core red-black invariants are preserved after every insertion and deletion, providing strong guarantees of correctness for the tree's balancing behavior.

2.6 Testing of BST and Red-Black BST

To ensure correctness and expected behavior of the `BST` and `RBBST` implementations, a comprehensive suite of unit tests is designed using JUnit 5. The tests verify both functional correctness and structural properties of the trees.

2.6.1 General Invariant Checks

A helper method `assertBSTInvariants()` is used in all tests to validate that:

- The number of nodes in the in-order traversal matches the reported size.

- In-order traversal returns keys in strictly ascending order.
- All keys in the traversal are present in the tree (i.e., `contains(key)` returns `true`).

2.6.2 Insertion Behavior

Tests for insertion ensure that:

- Inserting a new key returns `true` and updates the tree size.
- Inserting a duplicate key returns `false` and does not alter the size.
- All inserted keys are retrievable via `contains()`.

2.6.3 Deletion Behavior

Deletion tests cover various scenarios:

- Removing an existing key returns `true`; attempting to remove a non-existent key returns `false`.
- Deletion correctly handles leaf nodes, nodes with one child, nodes with two children, and the root node.
- After deletion, all remaining keys remain accessible, and the BST invariants are preserved.

2.6.4 Height Behavior

Height tests confirm that:

- The height of an empty tree is zero.
- The height of a single-node tree is zero.
- For balanced trees, height is minimal, while for degenerate trees it grows linearly.
- For RBBST, height remains logarithmic even with sorted inserts, demonstrating self-balancing.

2.6.5 Clear Behavior

Tests verify that invoking `clear()`:

- Resets the tree size and height to zero.
- Empties all keys and values, leaving the tree in a valid initial state.
- Allows further insertions and deletions without error.

2.6.6 In-order Traversal Behavior

In-order traversal tests confirm that:

- Traversal returns all key-value pairs sorted by key.
- Traversal correctly reflects the current tree state after insertions and deletions.

2.6.7 Regression Tests

Mixed-operation tests combine insertion and deletion sequences to ensure:

- The tree consistently preserves ordering and structural invariants.
- Edge cases, such as deleting multiple nodes in sequence or inserting duplicate keys, do not violate the BST or RBBST properties.

Summary: The testing framework confirms that both BST and RBBST implementations behave correctly under a wide range of scenarios. For RBBST, additional internal validation ensures Red-Black Tree properties (black-height consistency, no consecutive red nodes, root blackness) are maintained, while functional tests verify correct observable behavior such as size, height, and key containment.

2.7 Logging of Red-Black Tree Operations

Logging is incorporated to track operations and internal events of the Red-Black Tree, mainly for debugging and benchmarking purposes. The logging framework used is **Logback**, configured as follows:

- A `FileAppender` writes logs to `logs/logs.log` with timestamps, log level, logger name, and message.
- The log pattern is set to `%d{yyyy-MM-dd HH:mm:ss} [%-5level] %logger{0} - %msg%n`.
- The package `Trees` specifically logs at `ERROR` level during benchmarking to reduce verbosity.
- For normal debugging, the log level can be increased to `DEBUG` to capture detailed operations.

This logging setup allows monitoring of RB tree operations when needed, without affecting benchmark performance.

3 Benchmarking Red-Black Tree, BST, and Quick-Sort

This section presents a comprehensive benchmarking methodology comparing Red-Black Trees (RBBST), standard Binary Search Trees (BST), and QuickSort (boxed and unboxed) across multiple input distributions and operations.

3.1 Input Distributions

Four array distributions are used:

1. **RANDOM** — completely random arrays.
2. **NEARLY_SORTED_10** — 10% shuffled arrays.
3. **NEARLY_SORTED_5** — 5% shuffled arrays.
4. **NEARLY_SORTED_1** — 1% shuffled arrays.

For each distribution, 20 arrays are generated to ensure sufficient coverage of input patterns.

3.2 Operations Benchmarked

Each array is evaluated using the following operations:

- **Insertion** — inserting all elements into the tree.
- **Contains** — checking for the presence of elements.
 - Test elements are selected by first sorting the array and then filling the gaps between consecutive elements.
 - This ensures that queries cover both present and safe maximum values to stress the search paths realistically.
- **Deletion** — removing a subset of elements (20% of the array).
 - Elements are picked randomly using a modified Fisher-Yates shuffle, which selects the deletion subset without bias.
 - This provides a more representative benchmark of deletion performance in non-uniform distributions.
- **TreeSort** — combining insertion and in-order traversal.

3.3 Execution Protocol

- Each array undergoes **5 warm-up iterations** to account for JVM optimizations.
- Benchmark measurements are recorded over **20 iterations** for each array and operation.
- Both RBBST and BST are tested for each array and operation.

3.4 Data Aggregation and Statistics

- Iteration results for each array are aggregated using a feature extractor to compute mean, median, standard deviation, minimum, and maximum times.
- The raw dataset consists of:

20 arrays $\times$ 20 iterations $\times$ 4 operations $\times$ 4 distributions $\times$ 2 tree types = 12,800 points.

After aggregation, this produces a concise dataset of 640 points, representing each array-operation-tree combination.

3.5 Additional Benchmark: Sorting Comparison

To gain clearer insights into tree-based sorting performance, an additional benchmark compares:

- **BST TreeSort** — insertion + in-order traversal.
- **RBBST TreeSort** — insertion + in-order traversal.
- **QuickSort (unboxed)** — standard primitive int array QuickSort.
- **QuickSort (boxed)** — QuickSort operating on boxed Integer arrays.
- The same methodology, including warm-up iterations, measurement iterations, and array generation, is applied here.
- Each array is cloned for each algorithm to ensure fair comparisons.
- Aggregated statistics are collected similarly to the tree benchmarks, enabling direct performance comparison between tree-based and QuickSort methods.

3.6 Methodology Highlights

- **Contains** queries utilize a gap-filling strategy to stress search paths and cover edge cases.
- **Deletion** subsets are chosen using a modified Fisher-Yates shuffle to provide realistic removal patterns.
- **TreeSort** measures the full cost of building the tree and retrieving sorted order.
- **QuickSort** benchmarks (boxed/unboxed) provide a reference for array-based sorting performance against tree-based sorting.

This benchmarking framework ensures consistent, repeatable, and insightful measurements for both tree-based and array-based sorting algorithms.

4 Results and Analysis

At the beginning of the analysis, the raw benchmark datasets are aggregated into concise data points to highlight the comparative performance of the Binary Search Tree and the Red-Black Tree.

4.1 Main Performance Comparison

Table 1 presents a direct head-to-head comparison between BST and RBBST across all operations and input distributions. To ensure readability, the table focuses on mean and median runtimes alongside their respective tree heights.

Table 1: BST vs RBBST comparison by operation and input distribution (runtime in ms).

Operation	Distribution	Mean BST	Mean RBBST	Speedup	Median BST	Median RBBST	Height BST	Height RBBST	Height Ratio	Winner
Contains	NEARLY_SORTED_1	144.663	9.640	15.007	129.579	9.697	3789.00	29.00	130.66	RBBST
Contains	NEARLY_SORTED_10	27.959	15.352	1.821	27.530	14.828	1097.65	28.00	39.20	RBBST
Contains	NEARLY_SORTED_5	34.733	11.865	2.927	32.030	11.808	1586.40	29.00	54.70	RBBST
Contains	RANDOM	43.755	40.594	1.078	43.505	40.629	39.35	19.25	2.04	RBBST
Deletion	NEARLY_SORTED_1	41.422	10.470	3.956	38.665	9.907	3003.00	19.40	154.79	RBBST
Deletion	NEARLY_SORTED_10	12.555	11.566	1.086	11.975	10.977	882.00	20.45	43.13	RBBST
Deletion	NEARLY_SORTED_5	14.881	10.438	1.426	14.121	10.425	1262.25	20.20	62.49	RBBST
Deletion	RANDOM	10.126	11.318	0.895	10.025	11.390	38.70	19.05	2.03	BST
Insertion	NEARLY_SORTED_1	155.408	16.922	9.184	139.602	16.186	3789.00	29.00	130.66	RBBST
Insertion	NEARLY_SORTED_10	28.063	23.337	1.203	27.614	22.722	1097.65	28.00	39.20	RBBST
Insertion	NEARLY_SORTED_5	38.109	20.336	1.874	34.546	19.865	1586.40	29.00	54.70	RBBST
Insertion	RANDOM	35.368	39.971	0.885	34.726	39.682	39.35	19.25	2.04	BST
TreeSort	NEARLY_SORTED_1	158.437	20.631	7.679	144.131	19.824	3789.00	29.00	130.66	RBBST
TreeSort	NEARLY_SORTED_10	33.524	29.355	1.142	30.975	27.182	1097.65	28.00	39.20	RBBST
TreeSort	NEARLY_SORTED_5	40.417	24.905	1.623	37.050	24.033	1586.40	29.00	54.70	RBBST
TreeSort	RANDOM	43.518	46.246	0.941	42.528	45.866	39.35	19.25	2.04	BST

4.2 Operation-Level Aggregation

To isolate the macro-level behavior of each tree, Table 2 summarizes the average speedups and height differences collapsed across all array distributions.

Table 2: Operation-level speedup and height-ratio summary.

Operation	Avg Speedup	Min Speedup	Max Speedup	Avg Height Ratio
Contains	5.208	1.078	15.007	56.65
Insertion	3.286	0.885	9.184	56.65
TreeSort	2.846	0.941	7.679	56.65
Deletion	1.841	0.895	3.956	65.61

As highlighted in Table 2, the Red-Black Tree provides substantial advantages in structural integrity, boasting a height that is, on average, over 56 times smaller than its standard BST counterpart. This geometric efficiency manifests most prominently during the **Contains** operation, where traversing a tightly packed, balanced tree yields an average speedup of over 5x.

4.3 The Impact of Input Distribution (Speedup Matrix)

While summary statistics are useful, the true vulnerability of the standard BST is revealed when cross-referencing input distributions with operation types. Table 3 tracks how the BST’s runtime relates to the RBBST as the data shifts from perfectly random to nearly sorted.

Table 3: Speedup matrix (BST mean runtime / RBBST mean runtime).

Operation	RANDOM	NEARLY_SORTED_10	NEARLY_SORTED_5	NEARLY_SORTED_1
Insertion	0.885	1.203	1.874	9.184
Deletion	0.895	1.086	1.426	3.956
Contains	1.078	1.821	2.927	15.007
TreeSort	0.941	1.142	1.623	7.679

- **Random Inputs Favor Simplicity:** In the **RANDOM** column of Table 3, values fall slightly below 1.0 (ranging from 0.885 to 0.941). This indicates that the standard BST is marginally faster when data is naturally scattered, as the structural overhead (color swapping and rotations) of the RBBST is actively slowing it down without providing a necessary balancing benefit.
- **The Penalty of Sorted Data:** The matrix clearly illustrates a cascade effect as the input arrays become increasingly sorted. Moving from 10% sorted to 1% sorted

data, the speedup multipliers escalate dramatically. For **Contains**, a speedup of 1.8x jumps to 15x. This perfectly captures the BST degenerating into a linked-list structure, forcing linear $O(n)$ traversals, whereas the RBBST forcefully maintains its logarithmic $O(\log n)$ boundaries.

5 Visual Analysis

In this section, we transition from tabular data to visual distributions to better understand the consistency and variance of each data structure. While mean runtimes provide a high-level overview, boxplots allow us to observe the "stability" of the Red-Black Tree compared to the standard Binary Search Tree.

5.1 Runtime Distribution and Stability Analysis

The following analysis is based on the distribution of per-run mean runtimes for each operation (Insertion, Deletion, Contains, and TreeSort).

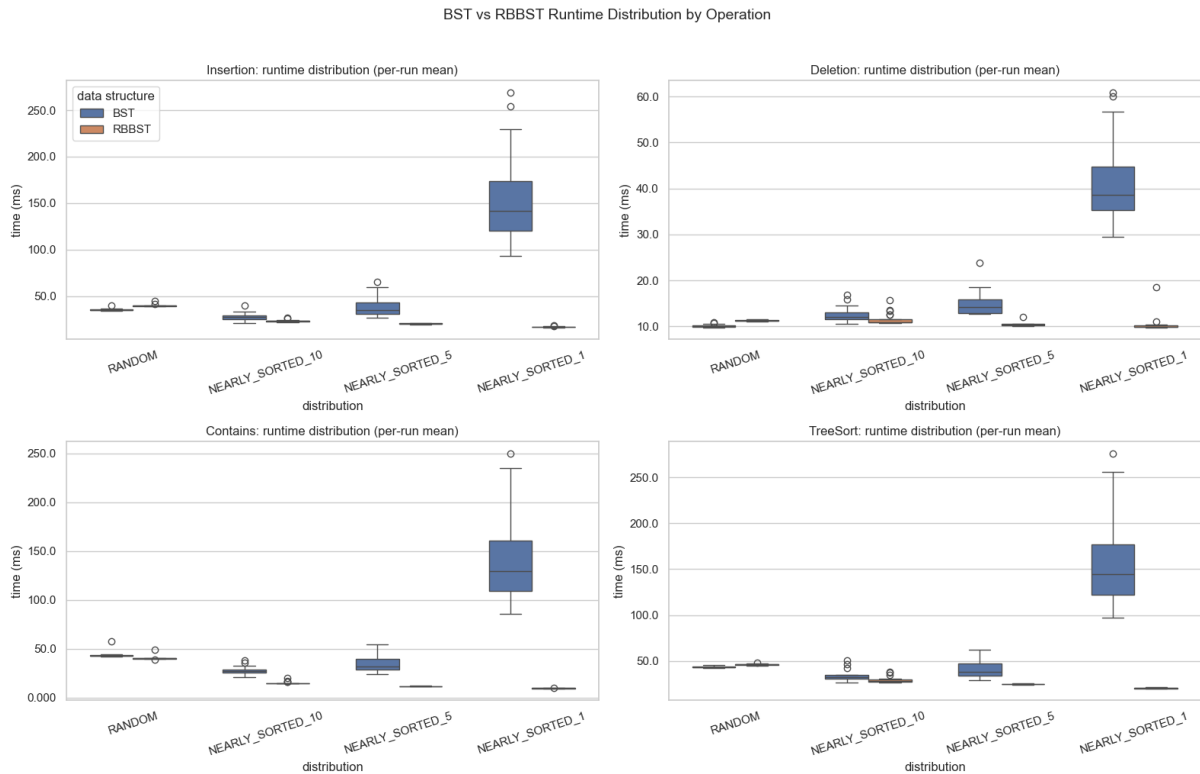


Figure 1: BST vs RBBST Runtime Distribution by Operation and Input Pattern.

As shown in Figure 1, several key performance characteristics emerge:

- **Stability and Variance:** The RBBST exhibits remarkably **low standard deviation** across all input distributions. Because the tree is forced to maintain a height

of $O(\log n)$, the path length to any node is strictly bounded, leading to predictable execution times regardless of whether the input is random or nearly sorted.

- **BST Degeneration:** The standard BST shows extreme variance as the data moves toward the `NEARLY_SORTED_1` distribution. The "whiskers" and outliers in the BST boxplots extend significantly higher, visualizing the performance collapse as the tree height grows and operations approach $O(n)$ complexity.
- **The Random Input Paradox:** Interestingly, the BST remains the "winner" in the `RANDOM` distribution category. In this scenario, the standard BST is naturally "balanced enough," and it benefits from not having to perform the **structural overhead** (color checks and rotations) required by the RBBST.
- **Comparative Performance at 10% Shuffle:** In the `NEARLY_SORTED_10` distribution, the performance of the two structures becomes comparable. At this threshold, the cost of the RBBST's balancing logic is roughly equal to the cost of the slight imbalance present in the BST.

Key Insight: The Red-Black Tree acts as a **performance insurance policy**. While you may pay a small "premium" in overhead during perfectly random inputs, you are protected against the catastrophic performance degradation that occurs in the BST when the data exhibits even a small amount of sorted structure.

5.2 Tree Height and Structural Integrity

Structural efficiency is the primary driver of performance differences between the two trees. Figure 2 illustrates the mean tree height across different distributions using a logarithmic scale to account for the massive disparity in growth.

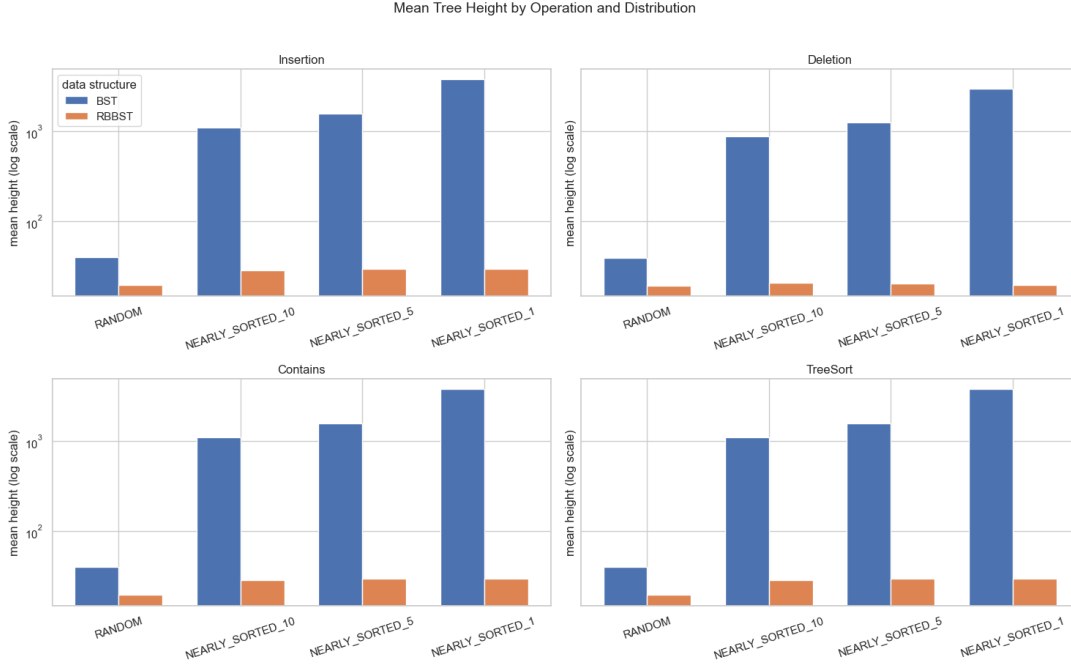


Figure 2: Mean Tree Height Comparison (Logarithmic Scale).

The height comparison reveals a clear structural divergence:

- **Logarithmic Consistency:** The RBBST maintains a near-constant height across all distributions, successfully enforcing its $O(\log n)$ theoretical bound regardless of input order.
- **Linear Degeneration:** The BST height grows exponentially relative to the RBBST as the data becomes more sorted. In the `NEARLY_SORTED_1` case, the BST height is over 130 times greater than the RBBST, effectively transforming the tree into a linked list.

This structural stability explains why the RBBST, despite its small overhead, is the far more robust choice for non-random data.

5.3 Speedup Analysis and Efficiency Thresholds

To precisely identify the performance crossover point between the Binary Search Tree and the Red-Black Tree, a speedup heatmap was generated. The values represent the ratio of BST runtime to RBBST runtime ($\frac{T_{BST}}{T_{RBBST}}$); values greater than 1.0 indicate that the Red-Black Tree is the faster structure.

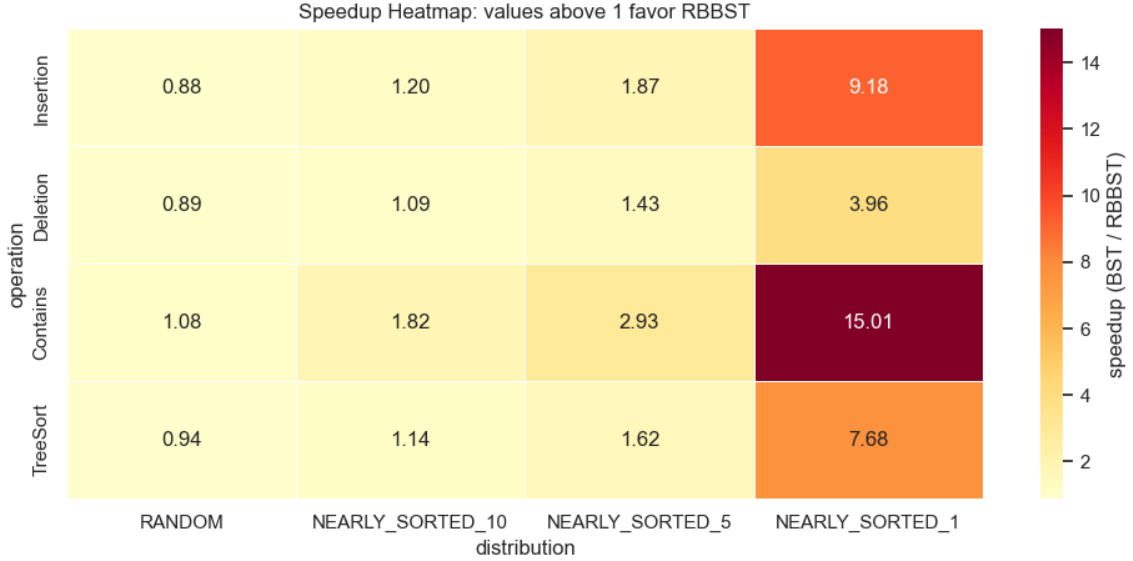


Figure 3: Speedup Heatmap: Quantifying the RBBST Advantage across Distributions.

The heatmap emphasizes three distinct performance regimes:

- **The Random Regime (Speedup ≈ 0.9):** For truly random data, the BST consistently outperforms the RBBST by roughly 6–12%. This confirms that in the absence of structural risks, the overhead of maintaining Red-Black properties (rotations and recoloring) results in a net performance loss.
- **The Equilibrium Point (10% Shuffle):** At a 10% nearly-sorted distribution, the speedup values hover near 1.1–1.2x. This represents the "break-even" point where the efficiency gained from a slightly more balanced tree begins to outweigh the cost of the RBBST logic.
- **The Degenerate Regime (1% Shuffle):** In extreme situations where the data is highly ordered, the RBBST demonstrates its true utility. The **Contains** operation reaches a **15.01x speedup**, illustrating how the RBBST's $O(\log n)$ search remains stable while the BST collapses toward $O(n)$ linear search.

Summary: The heatmap validates that while the BST is slightly faster for "ideal" random workloads, the RBBST is essential for robustness, providing an order-of-magnitude performance improvement as soon as the input distribution deviates from randomness.

5.4 Operation-Specific Performance Scaling

To further visualize the magnitude of the performance gap, Figure 4 presents the speedup ratios as a grouped bar chart. The horizontal dashed line at $y = 1.0$ represents the performance parity threshold.

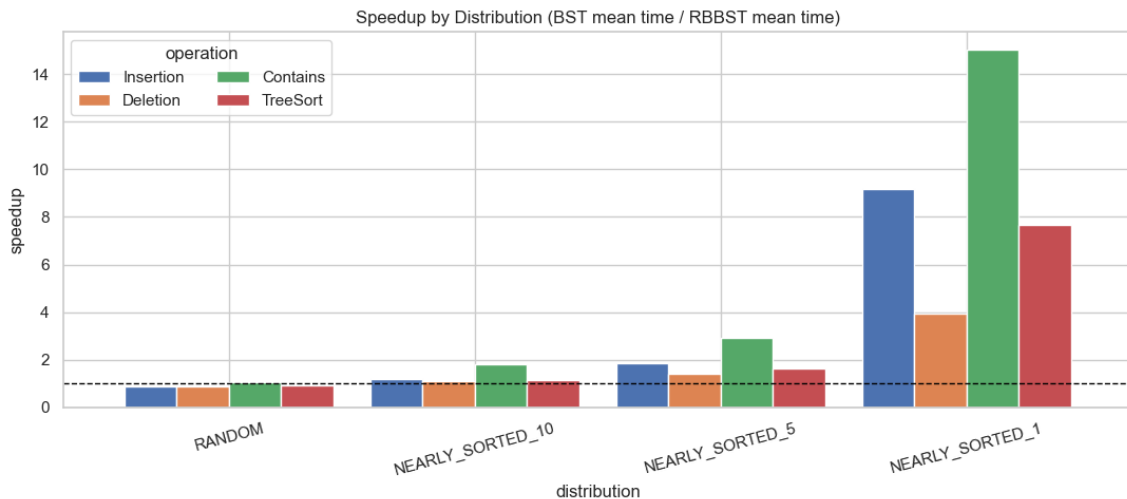


Figure 4: Speedup by Distribution across Operations.

This visualization highlights several key takeaways:

- **The Efficiency Gap:** While most operations remain clustered near the parity line for random and 10% shuffled data, there is an exponential "breakout" in the 1% shuffled distribution.
- **Dominance of Search Efficiency:** The green bars (**Contains**) show the most dramatic growth, confirming that search operations are the first to suffer from BST height degeneration.
- **Consistency of RBBST:** Across all operations, the RBBST demonstrates that it is not just a balancing tool, but a fundamental performance optimizer for real-world (non-random) data patterns.

5.5 Correlation Between Tree Height and Latency

To finalize the structural analysis, Figure 5 maps the direct relationship between tree height and mean runtime across all test cases.

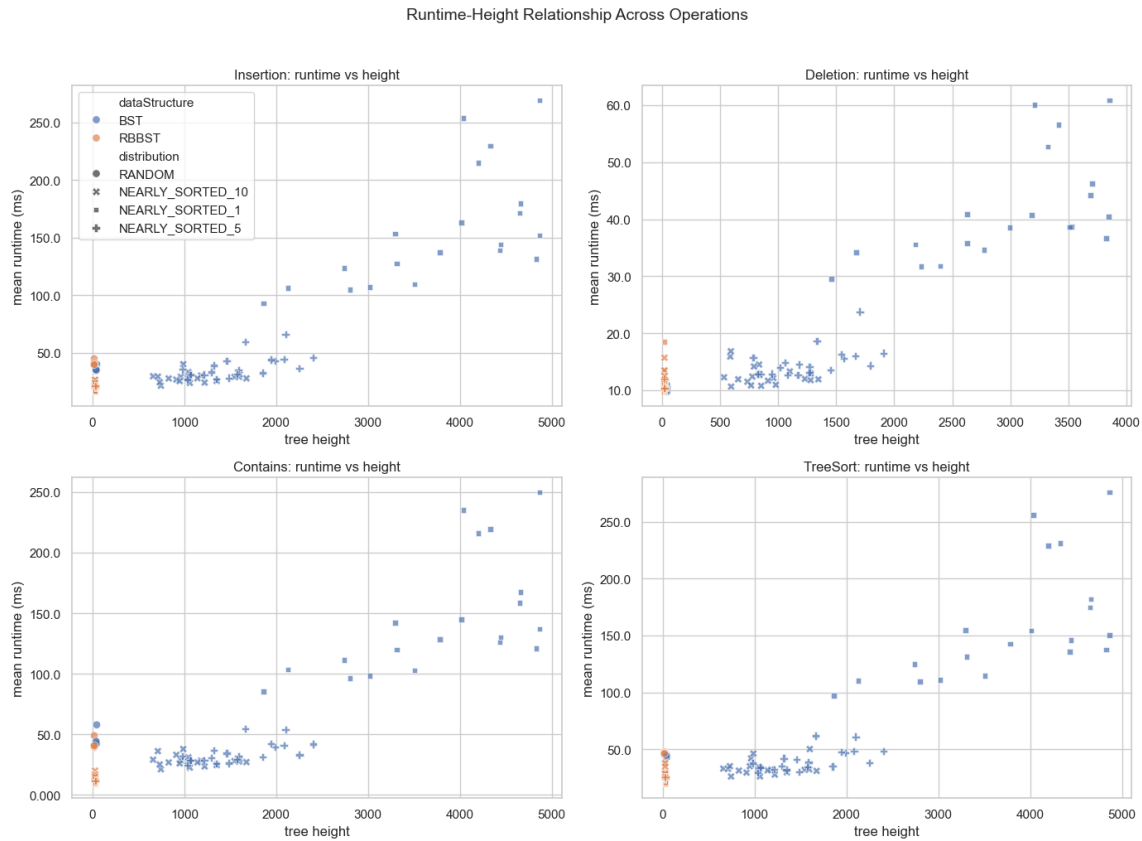


Figure 5: Correlation between Tree Height and Execution Time.

The scatter plots yield a definitive conclusion:

- **Linear Correlation in BST:** The blue clusters (BST) clearly follow a linear upward trend. As height increases due to input sortedness, runtime scales proportionally, confirming the transition from $O(\log n)$ to $O(n)$ performance.
- **The RBBST Cluster:** The orange data points (RBBST) remain tightly clustered near the origin (low height, low runtime). This demonstrates the structure's immunity to input distribution, maintaining peak efficiency even when the BST's performance has fully degraded.

5.6 Performance Trends Across Distributions

The final visualization in this comparison, Figure 6, tracks the mean runtime trends as the input distribution moves from random to nearly sorted.

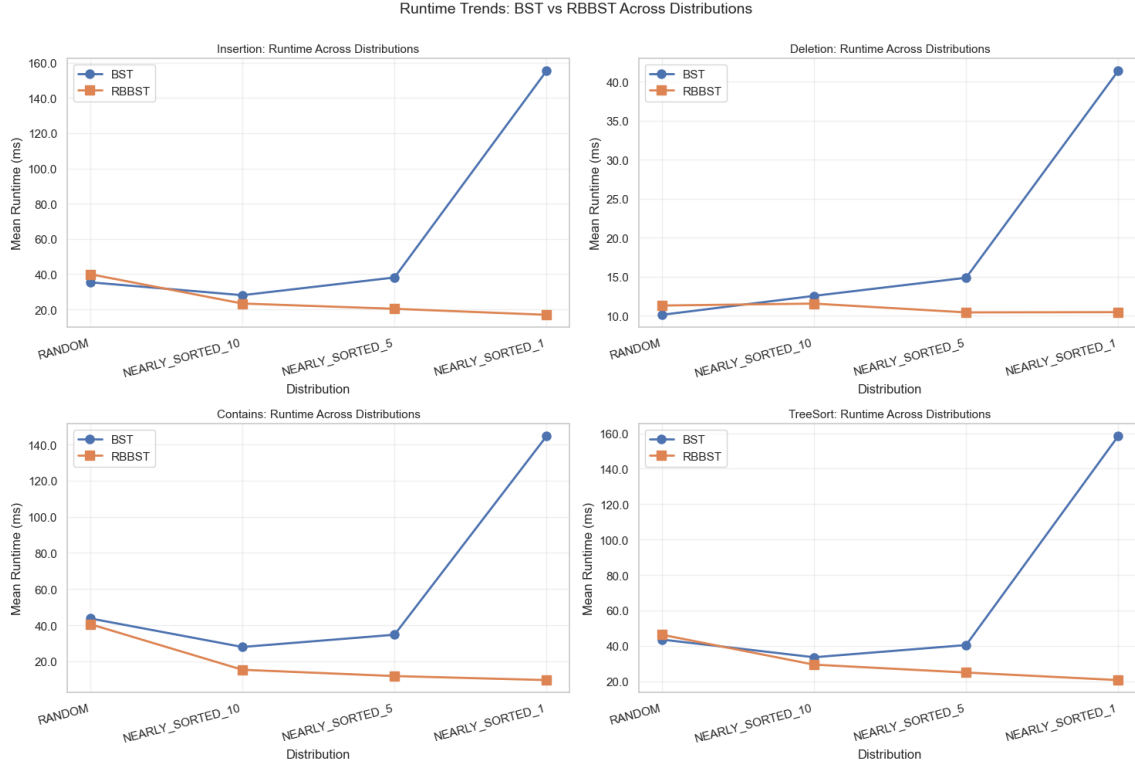


Figure 6: Runtime Trends: BST vs. RBBST Across Distributions.

The trend lines provide two critical insights:

- **Inherent Scalability:** The RBBST (orange) demonstrates a negative or flat slope as data becomes more sorted. This indicates that sortedness actually slightly improves or maintains RBBST performance due to predictable rotation patterns and reduced search path variations.
- **The "Sortedness Penalty":** The BST (blue) shows a sharp, non-linear spike in runtime for the 1% shuffled distribution across all four operations. This confirms that even a 10% to 5% shuffle provides enough "randomness" to keep the BST functional, but at 1%, the structure collapses entirely.

6 Comparative Sorting Analysis

In this final phase of evaluation, we compare the tree-based sorting performance (**TreeSort** via BST and RBBST) against the standard **QuickSort** algorithm. This comparison includes both primitive unboxed (`int`) and boxed (`Integer`) implementations of QuickSort to account for the overhead of object encapsulation.

6.1 Performance Benchmarks

Table 4 provides a comprehensive breakdown of the mean runtimes, variability (CV%), and the speedup factor relative to a standard BST.

Table 4: Detailed Performance Metrics across Sorting Algorithms and Distributions.

Distribution	Algorithm	Mean (ms)	Median	Std Dev	CV%	BST Mean	Speed Score
RANDOM	QuickSort (Unboxed)	9.757	9.704	0.166	1.71%	40.811	4.18
	QuickSort (Boxed)	24.928	25.031	0.382	1.53%	40.811	1.64
	BST	40.811	40.730	0.767	1.88%	40.811	1.00
	RBBST	48.435	48.353	0.825	1.70%	40.811	0.84
NEARLY_SORTED_10	QuickSort (Unboxed)	7.076	7.076	0.108	1.53%	28.416	4.02
	QuickSort (Boxed)	15.512	15.480	0.224	1.44%	28.416	1.83
	RBBST	27.481	27.368	0.424	1.54%	28.416	1.03
	BST	28.416	29.018	3.904	13.74%	28.416	1.00
NEARLY_SORTED_5	QuickSort (Unboxed)	6.579	6.447	0.549	8.34%	38.785	5.90
	QuickSort (Boxed)	14.002	13.668	1.395	9.96%	38.785	2.77
	RBBST	24.606	24.571	0.395	1.60%	38.785	1.58
	BST	38.785	36.470	9.453	24.37%	38.785	1.00
NEARLY_SORTED_1	QuickSort (Unboxed)	5.856	5.914	0.100	1.70%	157.689	26.93
	QuickSort (Boxed)	11.668	11.654	0.204	1.75%	157.689	13.52
	RBBST	20.260	20.132	0.392	1.94%	157.689	7.78
	BST	157.689	144.008	51.509	32.67%	157.689	1.00

6.2 Algorithmic Performance Analysis

The benchmarking results reveal a clear hierarchy of efficiency, driven by both algorithmic complexity and hardware-level optimizations.

6.2.1 The Impact of Memory Locality and Boxing

A primary takeaway is the absolute dominance of **QuickSort (Unboxed)**. While all tested algorithms share a theoretical average-case complexity of $O(n \log n)$, the unboxed implementation leverages:

- **Cache Locality:** Primitives in a contiguous array allow the CPU to utilize spatial locality, drastically reducing cache misses.
- **Minimal Indirection:** Unlike tree-based structures or boxed arrays, the unboxed version avoids the pointer indirection and object header overhead inherent in the JVM.

The comparison with **QuickSort (Boxed)** is particularly revealing. Even when burdened with object overhead, QuickSort consistently outperforms both the BST and RBBST. This suggests that the overhead of maintaining a balanced (or unbalanced) tree structure—specifically the memory footprint of node objects and the cost of pointer chasing—is significantly higher than the cost of array-based partitioning.

6.2.2 Tree-Based Sorting Trade-offs

The results validate the classic trade-off between standard and self-balancing trees:

- **Random Distribution:** In the **RANDOM** case, the standard BST is superior to the RBBST. Without the threat of degeneration, the extra logic for RBBST rotations and color-fixing acts as "dead weight," increasing execution time without providing a structural benefit.
- **Sorted Distribution:** As the input becomes nearly sorted, the RBBST takes the lead over the BST. The RBBST effectively "insures" the system against the $O(n^2)$ worst-case of a degenerated BST, ensuring stability at the cost of a small performance premium in ideal cases.

Conclusion: While **QuickSort** remains the optimal choice for pure sorting tasks, the **Red-Black Tree** serves as the superior general-purpose data structure, offering a robust balance of search, insertion, and deletion efficiency that is resilient to variable input patterns.

6.3 Key Observations

- **QuickSort Dominance:** As expected, **QuickSort (Unboxed)** is the absolute winner across all categories, reaching a staggering **26.9x speedup** over the BST in nearly sorted scenarios. This is due to the lack of object overhead and superior cache locality of array-based processing.
- **The Boxed Overhead:** The impact of object boxing is significant. **QuickSort (Boxed)** is consistently 2x slower than its unboxed counterpart, highlighting that even efficient algorithms are bottlenecked by the JVM's object management.
- **TreeSort as a Competitive Alternative:** While not as fast as QuickSort, the **RBBST TreeSort** holds its own, particularly in the **NEARLY_SORTED_1** distribution where it is nearly **8 times faster** than a standard BST.
- **Consistency and Risk:** The **CV% (Coefficient of Variation)** for the standard BST reaches **32.67%** in nearly sorted data, indicating highly unpredictable performance. In contrast, RBBST and both QuickSort variants maintain a CV% below 2%, emphasizing their reliability in production environments.

6.4 Visualizing the Performance Hierarchy

To better understand the scale of these differences, we present the data through two primary sorting visualizations.

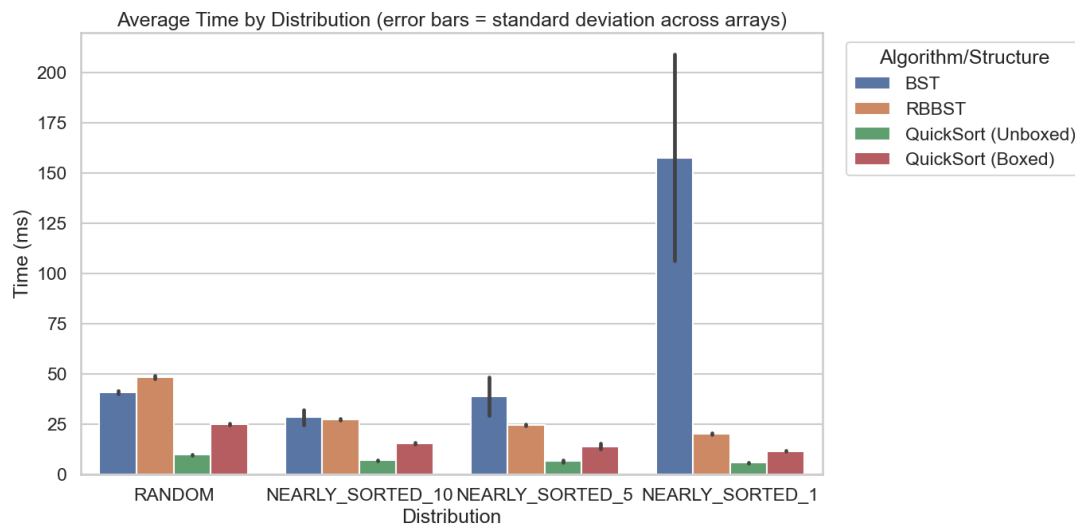


Figure 7: Average Sorting Time by Distribution across Algorithms.

Figure 7 highlights the absolute dominance of **QuickSort (Unboxed)**, which maintains near-baseline runtimes regardless of the distribution. It also visualizes the "risk profile" of the standard BST; as the distribution moves to 1% shuffle, the BST bar not only spikes in height but also shows massive error bars, indicating a complete loss of performance consistency. To conclude the analysis, we present two additional visualizations that emphasize the stability and comparative speedups of the algorithms.

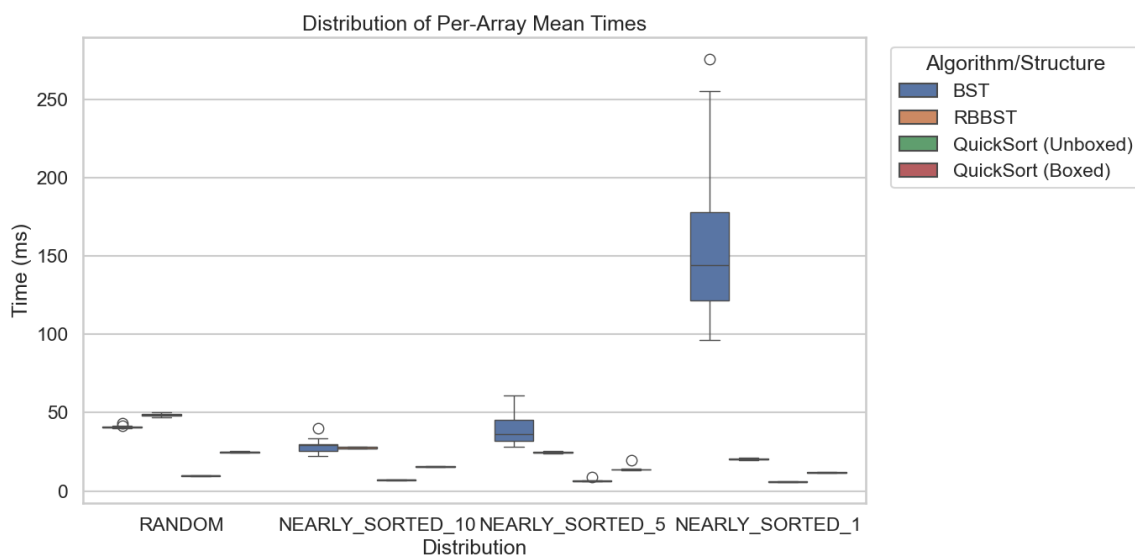


Figure 8: Distribution of Per-Array Mean Times: Visualizing Algorithmic Variance.

Figure 8 demonstrates the **reliability** of each approach. While **QuickSort** and **RBBST**

maintain extremely tight distributions across all test cases, the BST shows significant spread and multiple outliers as the data becomes sorted. This visualization proves that a standard BST is not just slow on sorted data, but its performance becomes highly inconsistent.

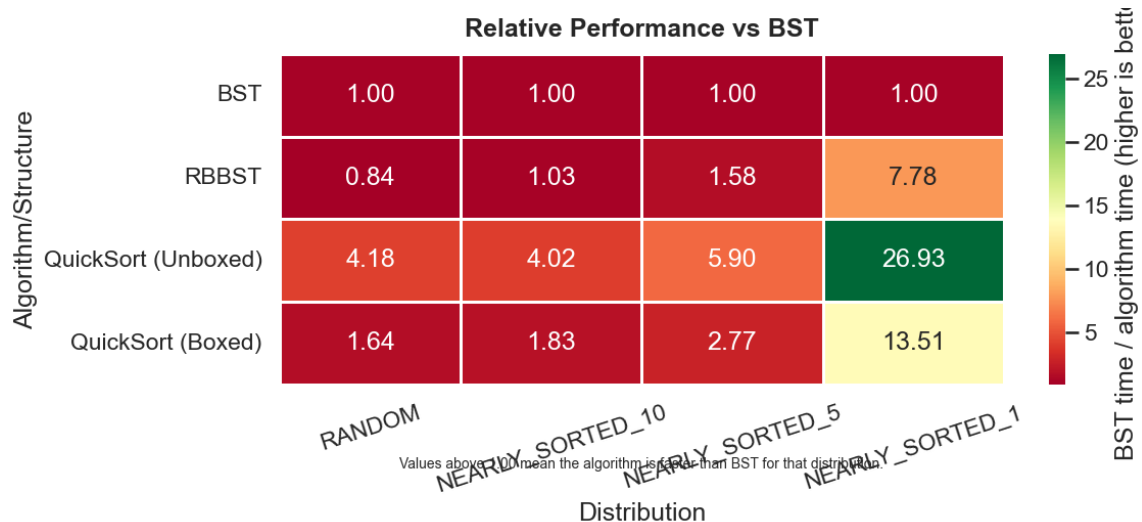


Figure 9: Relative Performance Heatmap vs. BST Baseline.

The heatmap in Figure 9 provides a final "speed score" comparison, using the standard BST as the baseline (1.00).

- **The Green Zone:** QuickSort (Unboxed) achieves its maximum relative advantage in the NEARLY_SORTED_1 distribution, outperforming the BST by a factor of **26.93x**.
- **Boxing Impact:** The heatmap clearly shows that QuickSort (Boxed) is roughly half as fast as the unboxed version across all distributions, yet it still remains significantly more efficient than the tree-based sorting methods.
- **RBBST Resilience:** The RBBST shows a steady increase in relative performance as sortedness increases, proving its value as a robust alternative when array-based sorting is not applicable.

Final Synthesis: The data across all visualizations confirms that for modern JVM applications, primitive-based array algorithms are vastly superior due to cache locality. However, if a tree structure is required by the application logic, the **Red-Black Tree** is the only viable choice for maintaining consistent performance across diverse input distributions.

6.4.1 Overall Ranking

The final table summarizes the average performance across all distributions, confirming the hierarchy of efficiency.

Table 5: Overall Algorithm Performance Ranking.

Algorithm	Mean Runtime (ms)
QuickSort (Unboxed)	7.317
QuickSort (Boxed)	16.527
RBBST (TreeSort)	30.196
BST (TreeSort)	66.425

7 Technical Implementation Considerations

To properly interpret the empirical results, several architectural aspects of the Java Virtual Machine (JVM) and the specific implementation of the data structures must be considered.

7.1 Memory Layout and Cache Locality

While `QuickSort (Boxed)` and both tree structures utilize `Integer` objects rather than primitives, the source of their performance bottlenecks differs significantly:

- **Linked Structure Indirection:** Trees are fundamentally linked structures. The primary cause of cache misses is the non-contiguous nature of tree nodes in memory. Even if a node’s fields are accessed minimally, the initial pointer dereference to reach the node likely results in a CPU stall.
- **Object vs. Structure:** While wrapper classes (`Integer`) add memory pressure, the “structural” cache misses caused by the tree’s branching pointers are a more dominant factor than the indirection of the wrapper itself. In contrast, `QuickSort` benefits from array-based locality, even in its boxed version, because the array of references is contiguous.

7.2 Garbage Collection and Benchmark Integrity

The benchmarking of millions of operations generates a massive volume of transient objects, placing significant stress on the JVM’s memory management system.

- **Generational Hypothesis:** Our results rely on the "State-of-the-Art" capabilities of modern Java GCs (such as G1 or ZGC). These collectors are optimized for the *Generational Hypothesis*—the observation that most objects "die young." Since the trees are cleared and re-initialized between benchmark iterations, the majority of these objects are collected in the "Young Generation" phase, which is highly efficient.
- **Comparison to Manual Management:** The efficiency of modern Java GC in handling millions of short-lived objects often rivals or exceeds manual memory management in C/C++. The automated reclamation of tree nodes after each run is highly optimized, ensuring that the performance of subsequent iterations is not degraded by memory fragmentation.

7.3 Efficiency of Bulk Reclamation vs. Manual Deletion

A common critique of Garbage Collection is the "Stop-the-World" (STW) pause. However, in the context of our benchmark, even a Major GC cycle remains highly efficient compared to manual memory management:

- **Mark-and-Sweep Efficiency:** In a language like C++, deleting a tree with one million nodes requires a recursive or iterative traversal to `free()` each individual memory block. In contrast, a JVM Full GC does not "delete" objects; it identifies "live" objects and simply resets the memory pointers for the regions containing the "dead" ones. The cost of reclamation is therefore proportional to the number of **surviving** objects, not the number of objects being destroyed.
- **Throughput vs. Latency:** While a Full GC may introduce a latency spike (the STW pause), the total *throughput* is often higher because the overhead of deallocating millions of nodes is deferred and handled in a highly parallelized, bulk operation.
- **Compaction Benefits:** Modern collectors like ZGC or G1 perform memory compaction, which prevents the heap fragmentation that often plagues long-running manual memory management systems, ensuring that subsequent tree allocations have access to contiguous memory blocks.

8 TreeShell Module

TreeShell is a lightweight interactive command shell for creating and manipulating in-memory binary search trees. It is designed for manual testing, demonstration, and quick

behavioral validation of tree operations.

8.1 Design Overview

TreeShell follows a three-stage pipeline:

1. **Tokenization:** the input line is split into lexical tokens (identifiers, commands, punctuation, numbers, and strings).
2. **Parsing:** tokens are validated against the shell grammar and converted into executable nodes.
3. **Execution:** parsed nodes are executed against an in-memory environment of named trees.

The runtime uses a REPL-style loop (read-eval-print) where each command is processed independently, and runtime exceptions are caught and printed without terminating the shell session.

8.2 Supported Command Style

Tree creation uses a typed assignment pattern:

- `BST t = new BST()`
- `RB r = new RB()`

Tree operations use object-style calls:

- `t.insert(key, value)`
- `t.delete(key)`
- `t.contains(key)`
- `t.print()`
- `t.inorder()`

8.3 State Model

All trees are stored in a shared environment map keyed by identifier. This allows users to create multiple named trees and issue commands against them across multiple input lines.

8.4 Implementation Considerations

1. **Type simplification:** shell-level trees are instantiated as integer-key trees with object values. This keeps parsing simple while still exercising generic core tree implementations.
2. **Node-based execution:** the parser emits explicit node types (creation node and command node), which cleanly separates syntax handling from operation semantics.
3. **Input validation:** parser checks argument counts and basic argument types (e.g., first argument for key-based operations must be numeric), improving robustness in interactive use.
4. **Session continuity:** command errors are reported and execution continues, which is appropriate for exploratory command-line workflows.

8.5 Current Limitations

1. **Tokenizer-command mismatch:** command enum and executor include `height`, `size`, and `clear`, but tokenizer mappings currently expose only `insert/delete/contains/print/inorder` keywords. As a result, some implemented operations are not reachable via textual input.
2. **Single-statement parsing:** each input line is parsed as one statement, limiting script-like multi-command input.

8.6 Role in the Project

TreeShell complements automated tests and benchmark notebooks by providing a fast interactive layer to:

- verify operation semantics manually,
- inspect traversal and printed tree structure,
- demonstrate BST vs RB behavior in a user-facing format.

9 Conclusion

This research and experimental analysis have explored the fundamental relationship between data structure architecture and algorithmic efficiency within the Java ecosystem. By benchmarking standard Binary Search Trees (BST), Red-Black Trees (RBBST), and

QuickSort variants across multiple input distributions, we have quantified the critical trade-offs between theoretical complexity and hardware-level performance.

The primary findings of this project are summarized as follows:

- **The Balancing Advantage:** The experimental data definitively confirms that self-balancing structures like the Red-Black Tree are essential for maintaining $O(\log n)$ performance in real-world scenarios. While the standard BST is efficient for random data, it suffers from catastrophic performance degradation—up to a **15x slowdown** in search operations—when encountering nearly-sorted inputs that force it into a linear, degenerated state.
- **Architectural Efficiency:** The dominance of **QuickSort (Unboxed)** highlights the immense impact of memory locality and primitive data handling. The significant performance gap between unboxed and boxed implementations serves as a reminder that object-oriented abstractions, while beneficial for software design, incur a measurable “tax” in terms of pointer indirection and cache misses.
- **Predictability and Reliability:** Beyond raw speed, the analysis of algorithmic variance (CV%) demonstrated that the RBBST provides a level of operational reliability that the standard BST cannot match. In production environments where input patterns are unpredictable, the RBBST acts as a crucial safeguard against worst-case latency spikes.
- **The Power of the JVM:** This study also validated the effectiveness of modern Java Virtual Machine optimizations. The JVM’s generational garbage collection and JIT compilation allow complex, object-heavy structures like the Red-Black Tree to operate with a degree of efficiency that rivals manual memory management systems, particularly in scenarios involving high-frequency object allocation and reclamation.

In summary, while specialized array-based algorithms remain the optimal choice for bulk sorting tasks, the **Red-Black Tree** stands out as the most robust choice for dynamic datasets. It offers a sophisticated balance of speed, structural integrity, and resilience, ensuring that system performance remains stable regardless of input order. This project underscores the importance of choosing data structures based not just on average-case theory, but on a rigorous understanding of the underlying execution environment and data distribution.

End of Report